

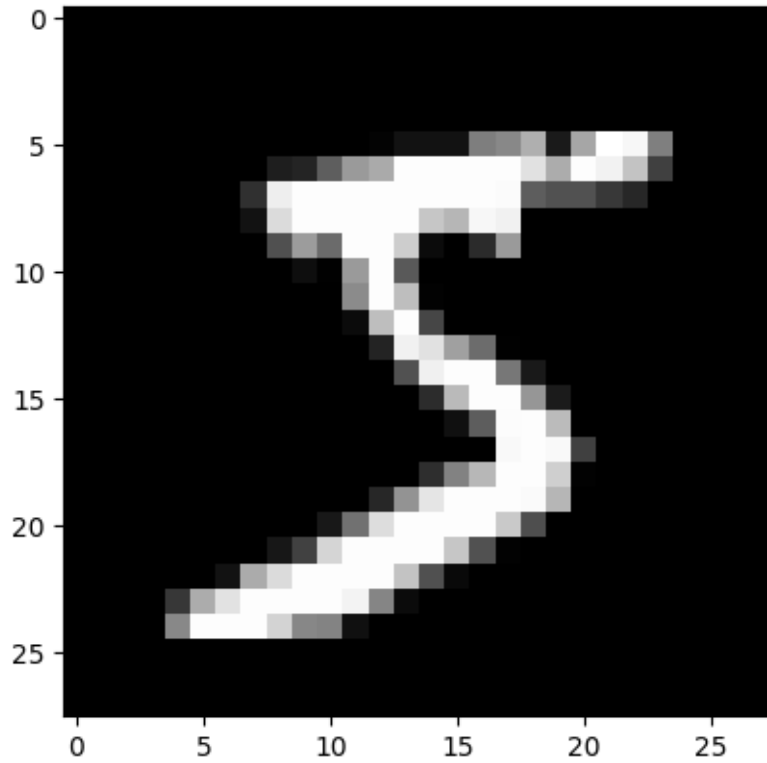
2B_Multiclass_Classification

May 9, 2026

```
[1]: import numpy as np
      from tensorflow.keras.models import Sequential
      from tensorflow.keras.layers import Dense, Dropout
      from tensorflow.keras.optimizers import RMSprop
      from tensorflow.keras.datasets import mnist
      import matplotlib.pyplot as plt
      from sklearn import metrics

[2]: # The MNIST dataset is a built-in dataset provided by Keras.
      # It consists of 70,000 28x28 grayscale images, each of which displays a single
      ↪handwritten digit from 0 to 9.
      # The training set consists of 60,000 images, while the test set has 10,000
      ↪images.
      (x_train, y_train), (x_test, y_test) = mnist.load_data()

[3]: plt.imshow(x_train[0], cmap='gray') # imshow() function which simply displays
      ↪an image.
      plt.show()
```



```
[4]: print(x_train[0])
```

```
[[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
   0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0  0  0  0  0  0  3  18  18  18 126 136
 175 26 166 255 247 127  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 30 36 94 154 170 253 253 253 253 253
 225 172 253 242 195 64  0  0  0  0]
 [ 0  0  0  0  0  0  0 49 238 253 253 253 253 253 253 253 253 251
 93 82 82 56 39  0  0  0  0  0]
 [ 0  0  0  0  0  0  0 18 219 253 253 253 253 253 198 182 247 241
  0  0  0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0 80 156 107 253 253 205 11  0 43 154
  0  0  0  0  0  0  0  0  0  0]
```

```

[ 0  0  0  0  0  0  0  0  0  0 14  1 154 253  90  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0 139 253 190  2  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  11 190 253  70  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  35 241 225 160 108  1
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  81 240 253 253 119
 25  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  45 186 253 253
150 27  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  16  93 252
253 187  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0 249
253 249 64  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  46 130 183 253
253 207  2  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  39 148 229 253 253 253
250 182  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0 24 114 221 253 253 253 253 201
 78  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0 23  66 213 253 253 253 253 198  81  2
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0 18 171 219 253 253 253 253 195  80  9  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  55 172 226 253 253 253 253 244 133  11  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0 136 253 253 253 212 135 132  16  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]
[ 0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
  0  0  0  0  0  0  0  0  0  0  0]

```

```

[5]: print("X_train shape", x_train.shape)
      print("y_train shape", y_train.shape)
      print("X_test shape", x_test.shape)
      print("y_test shape", y_test.shape)

```

```

X_train shape (60000, 28, 28)
y_train shape (60000,)
X_test shape (10000, 28, 28)
y_test shape (10000,)

```

```
[6]: x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)
x_train = x_train.astype('float32')
```

```
[7]: x_test = x_test.astype('float32')
x_train /= 255 # Each image has Intensity from 0 to 255
x_test /= 255
```

```
[8]: num_classes = 10
y_train = np.eye(num_classes)[y_train]
```

```
[9]: y_test = np.eye(num_classes)[y_test]
```

```
[10]: model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(784,)))
```

C:\Users\Lenovo\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
[11]: model.add(Dropout(0.2)) # DROP OUT RATIO 20%
model.add(Dense(512, activation='relu')) #returns a sequence of another vectors
↳ of dimension 512
model.add(Dropout(0.2))
model.add(Dense(num_classes, activation='softmax'))
```

```
[12]: model.compile(loss='categorical_crossentropy', # for a multi-class
↳ classification problem
optimizer=RMSprop(),
metrics=['accuracy'])
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

```
[13]: batch_size = 128 # batch_size argument is passed to the layer to define a batch
↳ size for the inputs.
epochs = 20
history = model.fit(x_train, y_train,
                    batch_size=batch_size,
                    epochs=epochs,
                    verbose=1, # verbose=1 will show you an animated progress bar eg.
↳ [=====]
                    validation_data=(x_test, y_test))
```

Epoch 1/20

469/469

11s 19ms/step -

accuracy: 0.9211 - loss: 0.2569 - val_accuracy: 0.9624 - val_loss: 0.1226
 Epoch 2/20
 469/469 8s 17ms/step -
 accuracy: 0.9676 - loss: 0.1048 - val_accuracy: 0.9762 - val_loss: 0.0759
 Epoch 3/20
 469/469 8s 17ms/step -
 accuracy: 0.9773 - loss: 0.0734 - val_accuracy: 0.9794 - val_loss: 0.0660
 Epoch 4/20
 469/469 8s 18ms/step -
 accuracy: 0.9818 - loss: 0.0577 - val_accuracy: 0.9799 - val_loss: 0.0676
 Epoch 5/20
 469/469 8s 18ms/step -
 accuracy: 0.9850 - loss: 0.0469 - val_accuracy: 0.9838 - val_loss: 0.0575
 Epoch 6/20
 469/469 11s 19ms/step -
 accuracy: 0.9870 - loss: 0.0408 - val_accuracy: 0.9824 - val_loss: 0.0654
 Epoch 7/20
 469/469 10s 18ms/step -
 accuracy: 0.9890 - loss: 0.0340 - val_accuracy: 0.9818 - val_loss: 0.0660
 Epoch 8/20
 469/469 10s 19ms/step -
 accuracy: 0.9908 - loss: 0.0292 - val_accuracy: 0.9846 - val_loss: 0.0603
 Epoch 9/20
 469/469 10s 18ms/step -
 accuracy: 0.9923 - loss: 0.0249 - val_accuracy: 0.9827 - val_loss: 0.0715
 Epoch 10/20
 469/469 10s 18ms/step -
 accuracy: 0.9926 - loss: 0.0231 - val_accuracy: 0.9840 - val_loss: 0.0620
 Epoch 11/20
 469/469 11s 19ms/step -
 accuracy: 0.9940 - loss: 0.0195 - val_accuracy: 0.9822 - val_loss: 0.0765
 Epoch 12/20
 469/469 8s 18ms/step -
 accuracy: 0.9942 - loss: 0.0193 - val_accuracy: 0.9844 - val_loss: 0.0667
 Epoch 13/20
 469/469 8s 17ms/step -
 accuracy: 0.9948 - loss: 0.0159 - val_accuracy: 0.9826 - val_loss: 0.0762
 Epoch 14/20
 469/469 9s 18ms/step -
 accuracy: 0.9950 - loss: 0.0149 - val_accuracy: 0.9838 - val_loss: 0.0715
 Epoch 15/20
 469/469 8s 18ms/step -
 accuracy: 0.9950 - loss: 0.0157 - val_accuracy: 0.9833 - val_loss: 0.0817
 Epoch 16/20
 469/469 9s 19ms/step -
 accuracy: 0.9955 - loss: 0.0138 - val_accuracy: 0.9826 - val_loss: 0.0825
 Epoch 17/20
 469/469 11s 23ms/step -

```
accuracy: 0.9961 - loss: 0.0121 - val_accuracy: 0.9852 - val_loss: 0.0752
Epoch 18/20
469/469          9s 19ms/step -
accuracy: 0.9964 - loss: 0.0107 - val_accuracy: 0.9847 - val_loss: 0.0772
Epoch 19/20
469/469          10s 18ms/step -
accuracy: 0.9966 - loss: 0.0108 - val_accuracy: 0.9850 - val_loss: 0.0755
Epoch 20/20
469/469          10s 21ms/step -
accuracy: 0.9968 - loss: 0.0099 - val_accuracy: 0.9863 - val_loss: 0.0792
```

```
[14]: score = model.evaluate(x_test, y_test)
      print('Test loss:', score[0])
      print('Test accuracy:', score[1])
```

```
Test loss: 0.07922611385583878
Test accuracy: 0.986299991607666
```

```
[ ]:
```